

API usage — Site Safety Compliance API

Kanagavel A K · RAP pre-hackathon screening · September 2026

Repository: <https://github.com/KanagavelAK/ppe-rt detr> · Live instance: <https://kanagavel-ppe-rt detr.hf.space> (Swagger at </api/docs>) · Weights: <https://github.com/KanagavelAK/ppe-rt detr/releases/download/v1.0/best.pt>

Site Safety Compliance API

RT-DETR fine-tuned on construction-site imagery to detect **helmet**, **head** (a bare head, no helmet) and **person**, served through FastAPI, with a hand-written reasoning layer that answers natural-language questions about an image and **refuses, with a reason, when the detections cannot support an answer**. Two of the three classes are outside COCO, so no off-the-shelf checkpoint can do this.

Live demo + API	https://kanagavel-ppe-rt detr.hf.space · Swagger at /api/docs
Weights	best.pt, 66 MB, GitHub release v1.0
Memo	memo/MEMO.pdf (2 pages) · source memo/MEMO.md
Reproduce training	notebooks/kaggle_ppe_rt detr.ipynb , Kaggle free tier, one Save & Run All
Metrics and receipts	artifacts/ — metrics.json , training_receipt.json , split_stats.json , failures/

Quickstart

Python 3.10 to 3.13 (tested on 3.10 in the Space, 3.11 in Docker, 3.12 on Kaggle, 3.13 locally). No GPU needed to serve.

```
git clone https://github.com/KanagavelAK/ppe-rt detr && cd ppe-rt detr
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --host 0.0.0.0 --port 8000      # downloads best.pt on first start
```

Swagger UI at <http://localhost:8000/docs>. Two held-out images are in [samples/](#). The first request after start-up is slow on CPU (kernel warm-up); later ones take about 0.6 s.

Docker: `docker build -t ppe-api . && docker run -p 8000:8000 ppe-api`

Demo page + API in one process, as deployed on the Space: `python space_app.py` → <http://localhost:7860>

Tests (no GPU or checkpoint needed): `python -m pytest tests -q` — 14 tests covering routing, the association rule and the guardrail.

Configuration

All settings are environment variables; `.env.example` lists them with defaults, and uvicorn picks up a `.env` automatically.

Variable	Default	Meaning
MODEL_WEIGHTS	artifacts/best.pt	checkpoint path
WEIGHTS_URL	GitHub release v1.0	where to fetch the checkpoint if the file is missing
WEIGHTS_KAGGLE_DATASET	empty	optional alternative source, owner/slug of a Kaggle dataset
CONF_THRESHOLD	0.25	default detection confidence cut-off (per-request override: form field confidence)
IMGSZ	640	inference size; must match training
DETECTOR_DEVICE	empty	empty lets Ultralytics choose; cpu pins inference to CPU
LOW_CONFIDENCE_FLOOR	0.45	guardrail: below this no assertion is made
ANTHROPIC_API_KEY	unset	optional; enables the language-model phrasing step
ANTHROPIC_MODEL	<code>claude-opus-5</code>	model for that single call
MAX_UPLOAD_BYTES	12582912	12 MB upload limit; larger files get a 400

Variable	Default	Meaning
LOG_LEVEL	INFO	one line per request with method, path, status, latency, request id

API

Real responses from the trained model on `samples/site_01.png`. On the live Space the paths are prefixed with `/api`.

POST /detect — image → boxes, labels, confidences

```
curl -X POST http://localhost:8000/detect -F "file=@samples/site_01.png" -F "confidence=0.25"
```

```
{
  "request_id": "261ae5f73801",
  "image": {"width": 416, "height": 415},
  "confidence_threshold": 0.25,
  "count": 4,
  "counts_by_class": {"helmet": 4},
  "detections": [
    {"label": "helmet", "confidence": 0.9073, "box_xyxy": [292.7, 103.0, 370.8, 197.1]},
    {"label": "helmet", "confidence": 0.9052, "box_xyxy": [75.7, 96.5, 159.3, 196.2]},
    {"label": "helmet", "confidence": 0.88, "box_xyxy": [291.1, 0.1, 374.2, 34.4]},
    {"label": "helmet", "confidence": 0.8747, "box_xyxy": [68.7, -0.1, 165.0, 41.8]}
  ],
  "inference_ms": 735.0
}
```

POST /ask — image + question → answer, or an honest refusal

```
curl -X POST http://localhost:8000/ask -F "file=@samples/site_01.png" \
-F "question=Is anyone not wearing a helmet?"
```

```
{
  "question": "Is anyone not wearing a helmet?",
  "answer": "Of 4 people detected, 4 are wearing a helmet and 0 are not.",
  "sufficient_information": true,
  "routing": {"needs_detection": true, "question_kind": "compliance",
    "rationale": "the question is about who is or is not wearing a helmet, which needs
per-person detections",
    "decided_by": "rules"},
  "detector_called": true,
  "guardrail_reasons": [],
  "answer_source": "template",
  "evidence": {
    "counts": {"helmet": 4}, "people_detected": 4, "person_boxes": 0,
    "compliant_workers": 4, "violations": 0, "undetermined_workers": 0,
    "workers": [{"id": 0, "status": "compliant", "headgear": "helmet", "headgear_confidence": 0.9
073}, "..."]
  },
  "inference_ms": 805.0
}
```

The three other kinds of answer, with the same fields:

Question	question_kind	detector_called	Answer
What is the capital of France?	not_about_the_image	false	That question is not about the contents of the image, so I did not run the detector.

Question	question_kind	detector_called	Answer
What colour is the truck?	out_of_detector_scope	false	I cannot answer that. This detector only recognises people, helmets and bare heads...
Is anyone not wearing a helmet? (<i>third worker's head occluded</i>)	compliance	true	I do not have enough information to answer that confidently. 1 of 3 detected people have no helmet and no bare head associated with them...

Every response carries routing, detector_called, guardrail_reasons and answer_source, so you can see why it answered or refused without logs. Errors return a typed {request_id, error, detail} with 400 (bad upload) or 500, and every response has an x-request-id header that matches the log line.

GET /health

```
{"status": "ok", "model_loaded": true, "weights": "...", "classes": [...], "llm_enabled": false}
```

Deployment

The live instance is a free Hugging Face Space (Gradio SDK, ZeroGPU tier). space_app.py mounts the FastAPI app inside a Gradio app, so one process serves the demo page at / and the API under /api/. Everything runs on CPU: ZeroGPU requires at least one @spaces.GPU function to start, so the file carries a placeholder that is never called, and no request reserves a GPU or consumes GPU quota. Free Spaces sleep after 48 h idle and wake in about a minute on the next request. The YAML block at the top of this file is the Space's configuration; to deploy your own copy, create a Space with SDK Gradio and push this repo to it.